

Project Final Report

EMG 기반 스마트 이동 제어 시스템 (NeuroMove)

교과목명: 캡스톤디자인

담당교수: 김웅섭

팀명: 삼박이일

팀원: 이연우 / 박수연 / 박수빈 / 박원

프론트엔드	https://github.com/Capstone-EMG-3P1L/NeuroMove_frontend
백엔드	https://github.com/Capstone-EMG-3P1L/NeuroMove_backend
AI	https://github.com/Capstone-EMG-3P1L/NeuroMove_AI
아두이노	https://github.com/Capstone-EMG-3P1L/NeuroMove_Arduino
동영상 자료	https://github.com/Capstone-EMG-3P1L/NeuroMove_Materials/releases/tag/v1.0.0

제출일자: 2026년 06월 15일

1. General Description

본 설계의 주제는 사용자의 근전도(EMG) 신호를 이용하여 이동 의도를 실시간으로 분석하고, 이를 기반으로 이동 보조 장치를 제어할 수 있는 스마트 이동 제어 시스템을 설계 및 구현하는 것이다.

시스템은 근전도 센서를 통해 사용자의 근육 신호를 수집하고, **ESP32**를 이용하여 신호를 디지털 데이터로 변환한 후 **AI** 서버로 전송한다. 이후 **AI** 분석 모듈에서는 전처리 과정을 거쳐 신호의 노이즈를 제거하고 유의미한 특징값을 추출하며, 머신러닝 기반 분류 모델을 이용하여 사용자의 의도를 **LEFT, RIGHT, STOP, REST**와 같은 명령으로 분류한다. 백엔드 서버는 사용자 정보, 캘리브레이션 결과, 디바이스 상태 및 운행 로그를 관리하고, **AI** 분석 결과를 기반으로 이동 장치 제어 명령을 생성한다. 또한 일정 시간 응답이 없거나 이상 패턴이 감지되는 경우 **STOP** 또는 **EMERGENCY_STOP** 명령을 우선 적용하여 **ESP32**와 안정적으로 연동되도록 한다.

특히 본 시스템은 단순한 신호 측정 시스템이 아닌 실시간 의도 추론 시스템이라는 점에 의의가 있다. 사용자의 근육 활동을 분석하여 이동 방향을 판단하고, 위험 상황에서는 정지 명령을 우선적으로 적용할 수 있도록 설계함으로써 실제 이동 보조 시스템에 적용 가능한 형태의 서비스를 목표로 한다.

또한 사용자별 근육 특성이 다르다는 점을 고려하여 캘리브레이션 기능을 제공하며, 사용자 맞춤형 제어 정책을 적용할 수 있도록 설계하였다. 이를 통해 다양한 사용자 환경에서도 안정적으로 동작할 수 있는 스마트 이동 제어 플랫폼을 구현하고자 한다.

2. Problem Statement

<성능 (처리량 및 반응시간)>

EMG 기반 제어 시스템은 사용자의 의도를 실시간으로 인식해야 하므로 높은 응답성이 요구된다. 사용자가 근육을 수축한 후 시스템이 이를 인식하고 제어 명령을 생성하기까지의 시간이 길어질 경우 사용성 저하뿐만 아니라 안전 문제로 이어질 수 있다.

특히 신호 수집, 전처리, 특징 추출, 머신러닝 추론, 제어 명령 생성 등 여러 단계가 연속적으로 수행되기 때문에 각 단계에서 발생하는 처리 지연을 최소화해야 한다. 따라서 본 설계에서는 경량 특징 추출 기법과 효율적인 추론 구조를 적용하여 실시간 처리가 가능하도록 설계하였다.

<데이터 (정확도 및 신뢰성)>

근전도 신호는 매우 민감한 생체신호로서 센서 부착 위치, 피부 상태, 사용자 근육 특성, 근육 피로도 등에 의해 큰 영향을 받는다. 또한 전원 노이즈나 주변 환경에 의한 잡음이 포함될 가능성이 높아 신호 품질이 저하될 수 있다.

이러한 문제는 의도 분류 정확도 저하로 이어질 수 있으므로, 신호 전처리를 통해 **DC Offset** 제거 및 정류(**Rectification**)를 수행하고 **MAV**, **RMS** 등의 특징을 추출하여 안정적인 입력 데이터를 생성한다. 또한 사용자별 캘리브레이션 과정을 통해 개인별 특성을 반영함으로써 데이터의 신뢰성과 분류 정확도를 향상시키고자 한다.

<경제성 (비용)>

기존의 상용 생체신호 기반 제어 장치는 고가의 의료 장비를 사용하는 경우가 많아 일반 사용자가 접근하기 어렵다. 또한 복잡한 하드웨어 구성은 유지보수 비용 증가의 원인이 될 수 있다.

본 설계에서는 비교적 저렴한 **MyoWare EMG** 센서와 **ESP32** 마이크로컨트롤러를 사용하여 시스템을 구축함으로써 비용 부담을 줄이고자 한다. 이를 통해 연구 목적뿐만 아니라 실제 상용화 가능성을 고려한 경제적인 시스템 구성을 목표로 한다.

<제어 및 보안>

이동 장치 제어 시스템은 사용자의 안전과 직접적으로 연결되므로 높은 수준의 안정성이 요구된다. 만약 시스템이 잘못된 의도를 인식할 경우 의도하지 않은 방향으로 이동하거나 충돌 사고가 발생할 수 있다.

이를 방지하기 위해 본 시스템은 특정 임계값 이하의 신호에 대해서는 **REST** 상태를 유지하도록 하며, **STOP** 신호가 감지되면 다른 명령보다 우선적으로 적용하도록 설계하였다.

또한 사용자 데이터와 제어 데이터가 네트워크를 통해 전송되는 과정에서 데이터 손실이나 변조가 발생하지 않도록 안정적인 통신 구조를 적용하고, **ESP32**와의 연결 상태 및 디바이스 상태를 관리하여 비정상 상황 발생 시 제어 명령이 안전하게 처리될 수 있도록 한다.

<효율성>

EMG 신호는 지속적으로 수집되기 때문에 많은 양의 데이터가 생성된다. 따라서 모든 데이터를 복잡한 알고리즘으로 처리할 경우 시스템 자원 사용량이 증가하고 응답성이 저하될 수 있다.

본 설계에서는 **MAV**(Mean Absolute Value)와 **RMS**(Root Mean Square)와 같은 계산 비용이 낮은 특징을 활용하여 효율적으로 신호를 표현하고, 머신러닝 모델 역시 경량화된 구조를 적용하여 임베디드 환경에서도 원활하게 동작할 수 있도록 설계하였다.

<서비스 (사용성 및 확장성)>

사용자가 시스템을 쉽게 사용할 수 있도록 직관적인 사용 환경을 제공하는 것도 중요한 설계 요소이다. 복잡한 설정 과정 없이 간단한 캘리브레이션만으로 시스템을 사용할 수 있어야 하며, 다양한 사용자 환경에서도 안정적으로 동작해야 한다.

향후 스마트 휠체어뿐만 아니라 재활 로봇, 의수·의족 제어 시스템, 스마트 헬스케어 기기 등 다양한 분야에 적용할 수 있도록 확장성을 고려하였다. 구체적인 서비스 확장 방향은 다음과 같다.

- 사용자별 근육 특성을 반영한 맞춤형 제어 기능 제공
- 근육 피로도 분석을 통한 사용자 상태 모니터링 기능 제공

- 스마트 휠체어 및 이동 보조 로봇과의 연동 기능 확장
- 실시간 생체신호 분석 기반 헬스케어 서비스 제공
- 사용자 데이터 기반 맞춤형 재활 프로그램 지원 기능 확장

3. Preliminary Study & Need for the Project

3.1. Background

최근 고령화 사회 진입과 장애인의 이동권 보장에 대한 사회적 관심이 증가함에 따라, 이동 보조 기술의 중요성이 더욱 부각되고 있다. 특히 전동휠체어와 같은 이동 보조 장치는 사용자의 독립적인 생활을 지원하는 핵심 수단으로 활용되고 있으나, 대부분의 제품이 조이스틱이나 버튼 기반의 입력 방식을 사용하고 있어 상지 근력이 부족하거나 손의 움직임이 제한된 사용자에게는 사용이 어렵다는 문제점이 존재한다.

실제로 근육 위축, 척수 손상, 신경계 질환 등의 이유로 손가락이나 손목의 정교한 움직임이 어려운 사용자는 기존 이동 보조 장치를 자유롭게 조작하기 어렵다. 이로 인해 이동의 자유가 제한될 뿐만 아니라 보호자의 도움에 의존하게 되는 경우가 많아 사용자의 자율성과 삶의 질이 저하되는 문제가 발생한다.

이러한 문제를 해결하기 위한 방법 중 하나로 생체신호를 활용한 인간-기계 인터페이스(Human-Machine Interface, HMI)가 주목받고 있다. 특히 근전도(EMG, Electromyography) 신호는 근육이 수축할 때 발생하는 전기적 신호를 측정하는 기술로, 사용자의 움직임 의도를 비교적 빠르고 정확하게 파악할 수 있다는 장점이 있다.

최근 센서 기술과 인공지능 기술의 발전으로 EMG 신호를 활용한 다양한 연구가 진행되고 있으며, 의료 재활 장비, 의수·의족 제어 시스템, 스마트 헬스케어 분야 등에서 활용 범위가 확대되고 있다. 그러나 기존 연구의 상당수는 고가의 장비를 사용하거나 특정 연구 환경에서만 동작하는 경우가 많아 실제 생활 환경에 적용하기에는 한계가 존재한다.

3.2. Current Scenario

현재 전동휠체어를 비롯한 대부분의 이동 보조 장치는 조이스틱이나 버튼과 같은 물리적 입력 장치를 통해 제어된다. 이러한 방식은 손과 팔의 정교한 움직임을 전제로 하기 때문에, 상지 근력이 약하거나 신경계 손상으로 손의 사용이 제한된 사용자에게는 접근성이 낮다는 한계가 있다. 한편 근전도(EMG) 신호를 활용한 제어 연구는 활발히 진행되고 있으나, 고가의 측정 장비와 폐쇄적인 연구 환경에 의존하는 경우가 많아 일상적인 사용 환경으로 확장되기 어려웠다.

본 프로젝트(NeuroMove)는 이러한 상황을 개선하기 위해 저비용 EMG 센서(MyoWare 2.0)와 ESP32 마이크로컨트롤러를 기반으로, 신호 수집부터 의도 추론, 이동 장치 제어까지 이어지는 전체 파이프라인을 직접 구현하였다. 시스템은 세 개의 독립된 저장소(repository)로 구성되며, 각각 AI 서버(Python/FastAPI), 백엔드 서버(Java/Spring Boot), 프론트엔드(React/TypeScript)를 담당한다.

현재 구현된 시스템은 ESP32가 3채널 EMG 신호를 WebSocket으로 AI 서버에 전송하면, AI 서버가 신호를 전처리하고 MAV, RMS 특징을 추출하여 RandomForest 모델로 LEFT, RIGHT, STOP, REST의 네 가지 의도를 분류한다. 분류 결과는 REST 상태 처리와 안전 우선 STOP 규칙을 거쳐 백엔드 서버로 전달되고, 백엔드는 위험도(Risk Score)와 페일세이프(fail-safe) 정책을 적용하여 최종 제어 명령을 결정한 뒤 모터 장치와 사용자 화면(웹 대시보드)에 실시간으로 반영한다. 즉, 단순 신호 측정을 넘어 실제 이동 제어로 이어지는 end-to-end 시스템이 동작하는 단계까지 구현이 완료되었다.

3.3. Challenges

본 시스템을 설계 및 구현하는 과정에서 다음과 같은 주요 기술적 과제가 존재하였다.

- 신호 품질과 개인차: EMG 신호는 센서 부착 위치, 피부 상태, 근육 피로도, 전원 노이즈 등에 민감하게 영향을 받으며, 사용자마다 신호의 크기와 분포가 크게 달라 동일한 임계값을 일괄 적용하기 어렵다.
- 실시간성: 신호 수집, 전처리, 특징 추출, 추론, 제어 명령 생성으로 이어지는 다단계 파이프라인의 지연을 최소화하여 사용자가 근육을 수축한 시점과 제어가 반영되는 시점의 간격을 짧게 유지해야 한다.
- 안전성: 잘못된 의도 인식은 의도하지 않은 이동이나 충돌로 직결되므로, 신호가 약하거나 비정상일 때 이동을 억제하고 STOP을 우선 적용하는 다중 안전 장치가 필요하다.
- 다중 서버 연동: AI 서버, 백엔드 서버, 프론트엔드, 임베디드 디바이스가 WebSocket과 REST API로 연결되므로, 서버 간 데이터 포맷 일치와 통신 안정성, 인증 처리가 중요하다.
- 디바이스 상태 관리: 하나의 ESP32 스트림을 캘리브레이션과 주행 세션에 모드 기반으로 분기해야 하며, 스트림이 비정상적으로 끊긴 디바이스의 점유(lock) 누수를 자동으로 회복해야 한다.

3.4. Need for the project

기존의 이동 보조 장치는 주로 조이스틱, 버튼, 수동 조작 방식에 의존하기 때문에 상지 사용이 제한된 사용자가 직관적으로 제어하기 어렵다는 한계가 있다. 또한 사용자의 신체 상태나 근육 사용 능력에 따라 동일한 제어 방식이 항상 적합하지 않을 수 있으므로, 사용자 개인의 생체신호를 반영한 맞춤형 이동 제어 기술이 필요하다.

본 설계에서는 이러한 한계를 보완하기 위해 저비용 EMG 센서와 ESP32 기반 임베디드 시스템을 활용하여 사용자의 근육 신호를 실시간으로 수집하고, 이를 분석하여 이동 의도를 추론하는 EMG 기반 스마트 이동 제어 시스템(NeuroMove)을 설계하고자 한다. 사용자가 특정 근육을 수축하면 시스템은 이를 분석하여 LEFT, RIGHT, STOP, REST와 같은 이동 명령으로 변환하고, 이를 이동 보조 장치의 제어 신호로 활용한다.

또한 단순한 신호 측정에 그치지 않고, AI 기반 의도 분류 기술과 실시간 제어 시스템을 결합하여 사용자의 의도를 보다 정확하고 안정적으로 반영할 수 있도록 설계하였다. 이를 위해 **EMG** 신호의 전처리, 특징 추출, 머신러닝 기반 의도 추론 과정을 통합하여 실시간으로 동작하는 제어 파이프라인을 구축하였다.

이를 통해 상지 사용이 제한된 사용자도 보다 쉽고 직관적으로 이동 장치를 제어할 수 있으며, 향후 스마트 휠체어, 재활 로봇, 웨어러블 헬스케어 기기 등 다양한 분야에 적용 가능한 기반 기술을 제시하는 것을 목적으로 한다. 또한 사용자의 자율성과 이동 편의성을 향상시키고, 생체신호 기반 인간-기계 인터페이스 기술의 실용화 가능성을 확인하는 데 의의가 있다.

4. Constraints

4.1. Technical Development Constraints

4.1.1 하드웨어 동작 환경

본 시스템은 사용자의 근전도(**EMG**) 신호를 측정하기 위해 **MyoWare 2.0 EMG** 센서와 **ESP32** 마이크로컨트롤러를 사용한다. **EMG** 센서는 3채널(**ch1, ch2, ch3**)로 구성되며 사용자 근육에서 발생하는 전기 신호를 측정한다. **ESP32**는 수집된 신호를 디지털 데이터로 변환하여 AI 서버로 전송하는 역할을 수행한다.

또한 모터 구동부는 백엔드 서버에서 결정된 최종 제어 명령을 실제 모터 동작으로 변환하기 위해 사용된다. 모터 제어 장치는 백엔드 서버로부터 전달된 **FORWARD, LEFT, RIGHT, STOP** 등의 명령을 수신하고, 모터 드라이버 보드를 통해 **DC** 모터의 회전 방향과 동작 상태를 제어한다.

정확한 **EMG** 신호 측정을 위해서는 센서 부착 위치와 사용자의 근육 상태가 일정하게 유지되어야 하며, 센서 접촉 상태나 외부 노이즈에 따라 신호 품질이 영향을 받을 수 있다. 또한 모터 구동부는 전원 공급 상태, 배선 안정성, 모터 드라이버의 동작 조건에 따라 제어 안정성이 달라질 수 있다.

4.1.2 AI 서버 동작 환경

AI 서버는 **Python** 기반으로 개발하며 **FastAPI** 프레임워크를 사용한다. 서버는 **ESP32**로부터 수신한 **EMG** 데이터를 실시간으로 처리하고, 신호 전처리와 특징 추출 과정을 거쳐 **RandomForest** 모델을 통해 사용자의 이동 의도를 추론한다.

실시간 처리가 요구되는 시스템 특성상 서버의 응답 시간이 전체 시스템 성능에 영향을 미치며, 동시에 여러 사용자가 접속할 경우 서버 자원 사용량이 증가할 수 있다.

4.1.3 백엔드 서버 동작 환경

백엔드 서버는 **Spring Boot** 기반으로 개발되며, 본 시스템에서 사용자 데이터 관리와 제어 명령 처리를 담당하는 중앙 서버 역할을 수행한다. 백엔드는 사용자 정보, 장치 정보, 세션 정보, 캘리브레이션 데이터, **AI** 추론 결과 및 명령 로그를 관리하며, 데이터 저장 및 조회를 위해 **MySQL**과 연동된다. 또한 실시간 세션 관리, 토큰 관리, 장치 연결 상태 관리 등 빠른 처리가 필요한 데이터는 **Redis**를 활용한다.

백엔드 서버는 **AI** 서버로부터 전달받은 추론 결과를 처리하고, 모터 제어 장치 및 **React** 기반 대시보드와 연동된다. 또한 실시간 제어 과정에서 발생할 수 있는 지연, 중복 명령, 연결 끊김 등에 대응하기 위해 **riskScore**, **FSM** 상태 관리, **fail-safe** 정책과 같은 안전 제어 로직이 동작할 수 있도록 구성된다.

4.1.4 시스템 통신 환경

본 시스템은 실시간 **EMG** 데이터 전송과 모터 제어를 위해 **WebSocket**과 **REST API**를 함께 사용한다. **ESP32**는 **MyoWare 2.0** **EMG** 센서에서 수집한 **EMG** 데이터를 **WebSocket**을 통해 **AI** 서버로 실시간 전송한다. **AI** 서버는 수신한 데이터를 분석하여 이동 의도, **confidence**, **signal quality**, **fatigue score** 등의 추론 결과를 생성하고, 이를 **REST API**를 통해 백엔드 서버로 전달한다.

백엔드 서버는 전달받은 추론 결과를 바탕으로 최종 제어 명령을 결정하고, **WebSocket**을 통해 모터 제어 장치로 명령을 전송한다. 또한 **React** 기반 대시보드는 백엔드 서버와 **WebSocket** 통신을 수행하여 **AI** 추론 결과, 모터 상태, 위험도, **FSM** 상태 등을 실시간으로 확인할 수 있도록 구성된다.

네트워크 지연이나 연결 끊김이 발생할 경우 실시간 의도 인식과 모터 제어 안정성에 영향을 줄 수 있다. 따라서 본 시스템은 **timestamp**, **WebSocket** 연결 상태 등을 활용하여 데이터 순서와 연결 상태를 확인할 수 있는 통신 환경을 구성한다.

4.2. Economic Constraints

<하드웨어 구매 비용>

EMG 신호 측정을 위한 **MyoWare** 센서와 **ESP32** 개발 보드가 필요하며, 센서 전극 및 연결 부품 등의 추가 비용이 발생한다. 프로젝트 규모에 따라 여러 개의 센서를 사용할 경우 비용이 증가할 수 있다.

<서버 운영 비용>

AI 서버와 백엔드 서버 운영을 위해 클라우드 서버 또는 테스트용 로컬 서버가 필요하다. 프로젝트 개발 단계에서는 로컬 환경을 활용하여 비용을 최소화하고, 향후 서비스 확장을 고려할 경우 클라우드 서버 비용이 발생할 수 있다.

<확장 비용>

향후 실제 스마트 휠체어 또는 이동 보조 장치와 연동할 경우 모터 제어 장치, 배터리 시스템, 추가 센서 등의 비용이 발생할 수 있다. 또한 사용자 수 증가에 따른 서버 증설 비용도 고려해야 한다.

4.3. Operational Constraints

<개발 및 유지보수 비용>

프로젝트 개발 과정에서 사용되는 대부분의 소프트웨어는 오픈소스 기반으로 구성되어 별도의 라이선스 비용은 발생하지 않는다. 다만 향후 실제 제품화 단계에서는 서버 운영 비용, 장비 유지보수 비용 및 사용자 지원 비용이 추가적으로 발생할 수 있다.

5. System Requirements

5.1. Functional Requirements

기능	설명
EMG 신호 수집	MyoWare 센서를 통해 사용자의 근전도 신호를 실시간으로 수집한다.
사용자 캘리브레이션	사용자별 근육 특성을 측정하여 기준값(Baseline) 및 임계값을 생성한다.
신호 전처리	DC Offset 제거, 정류(Rectification) 등을 수행하여 노이즈를 감소시킨다.
특징 추출	MAV, RMS 등의 특징값을 추출하여 AI 모델 입력 데이터로 사용한다.
의도 분류	AI 모델을 통해 LEFT, RIGHT, STOP, REST 의도를 분류한다.
실시간 데이터 전송	ESP32와 AI 서버 간 WebSocket 통신을 통해 데이터를 전송한다.
세션 관리	사용자별 세션 생성 및 종료 기능을 제공한다.
제어 결과 제공	분류된 의도를 이동 장치 제어 명령으로 변환하여 제공한다.
사용자 정보 관리	사용자 및 디바이스 정보를 관리한다.
데이터 저장 및 조회	캘리브레이션 결과, 세션 데이터, 추론 결과를 저장 및 조회한다.

5.2. Non-Functional Requirements

5.2.1 신뢰성 및 안전 요구사항

- 본 시스템은 사용자의 이동 의도를 기반으로 이동 장치를 제어하는 시스템이므로 높은 신뢰성과 안전성이 요구된다.
- 사용자의 의도가 명확하게 감지되지 않는 경우에는 이동 명령을 생성하지 않고 REST 상태를 유지하도록 한다.
- STOP 신호가 감지되는 경우 다른 이동 명령보다 우선적으로 처리하여 안전성을 확보한다.
- 센서 오류, 통신 장애 또는 비정상적인 신호가 발생하는 경우에는 이동 명령 생성을 중단하고 안전 상태로 전환한다.
- 사용자별 캘리브레이션 데이터를 활용하여 개인차에 따른 오동작을 최소화한다.
- 디바이스 연결 상태와 제어 명령 처리 상태를 관리하여 비정상 상황 발생 시 STOP 또는 EMERGENCY_STOP 명령을 우선 적용할 수 있도록 한다.

5.2.2 데이터 포맷

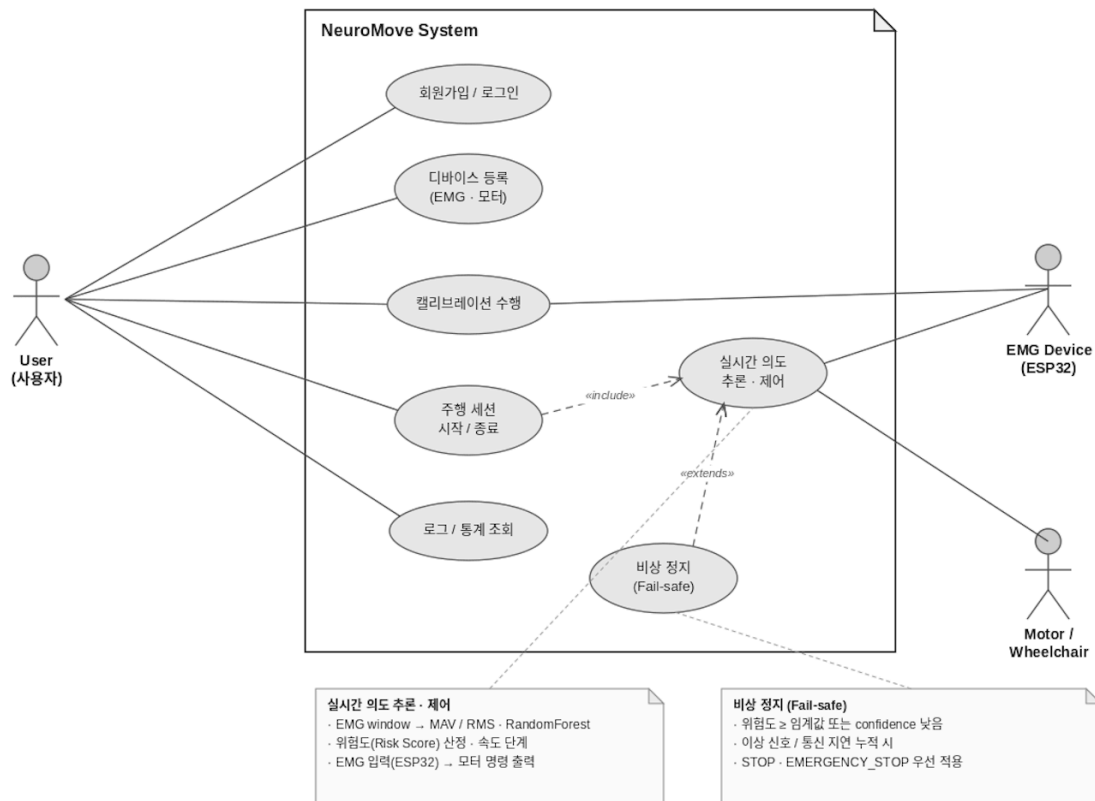
- EMG 데이터는 JSON 형식으로 구성하여 ESP32와 AI 서버 간 통신에 사용한다.
- AI 서버와 백엔드 서버 간 데이터 교환 또한 JSON 형식을 사용한다.
- 추론 결과는 Intent, Confidence, FatigueScore, SignalQuality 등을 포함하여 전송한다.
- 백엔드 서버는 수신된 추론 결과와 제어 명령, 운행 로그를 저장 및 관리할 수 있도록 데이터를 구조화한다.

5.2.3 입출력 방식

- 입력 데이터는 MyoWare EMG 센서를 통해 수집된 아날로그 신호를 ESP32에서 디지털 데이터로 변환하여 생성한다.
- ESP32는 WebSocket 통신을 이용하여 실시간으로 AI 서버에 데이터를 전송한다.
- AI 서버는 수신된 EMG 데이터를 전처리하고 특징을 추출한 후 AI 모델을 통해 의도를 분류한다.
- 분류 결과는 백엔드 서버로 전달되어 저장 및 관리되며, 백엔드 서버는 이를 기반으로 최종 제어 명령을 생성한다.
- 백엔드 시스템은 LEFT, RIGHT, STOP, REST 중 하나의 제어 결과를 생성하고, ESP32와의 연동을 통해 이동 보조 장치 제어에 활용할 수 있도록 설계한다.

6. System Design

6.1. Use Case Diagram



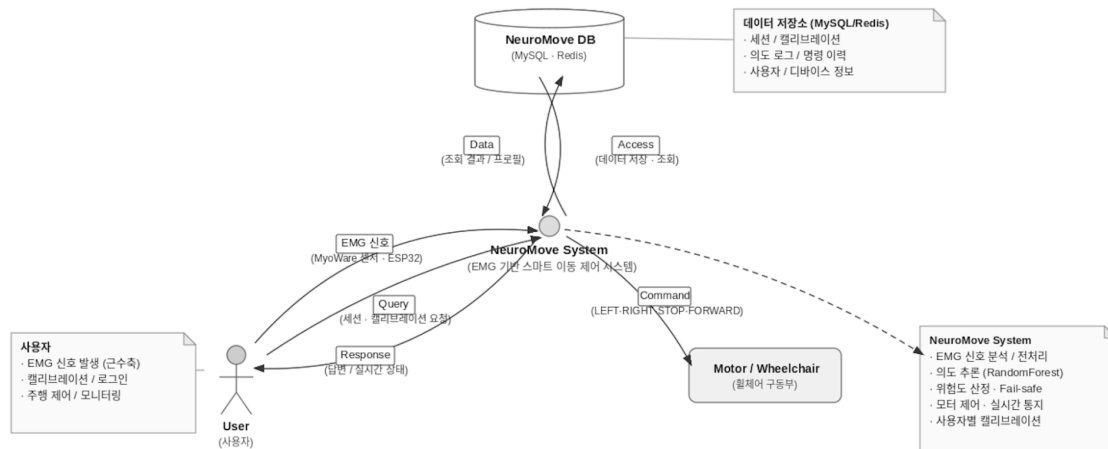
주요 액터

- 사용자(User): 회원가입, 디바이스 등록, 캘리브레이션, 주행 세션 제어, 로그 조회를 수행하는 주체
- EMG 디바이스(ESP32): EMG 신호를 수집하여 AI 서버로 실시간 전송하는 외부 액터
- 모터 디바이스(이동 보조 장치): 백엔드가 결정한 제어 명령을 수신하여 실제 구동을 수행하는 외부 액터

주요 유스케이스

- 회원가입 및 로그인: JWT 기반 인증을 통해 사용자 계정을 생성하고 접근 권한을 관리한다.
- 디바이스 등록 및 온보딩: 사용자가 보유한 EMG 디바이스와 모터 디바이스를 계정에 등록한다.
- 캘리브레이션 수행: REST, LEFT, RIGHT, STOP 단계별로 신호를 수집하여 사용자별 기준값(baseline)과 임계값을 생성한다.
- 주행 세션 시작 및 종료: 세션을 생성하여 실시간 의도 추론 및 제어를 활성화하고, 종료 시 통계를 집계한다.
- 실시간 의도 기반 제어: AI 추론 결과를 바탕으로 이동 명령을 생성하고 모터 디바이스에 전달한다.
- 로그 및 통계 조회: 세션별 의도 로그, 위험도, 명령 이력을 조회한다.

6.2. Data Flow Diagram



EMG 데이터는 다음과 같은 경로로 흐른다. ESP32는 3채널 EMG window 패킷(sequenceNumber, timestamp, samplingRate, windowSize, channels)을 JSON 형식으로 구성하여 AI 서버의 WebSocket 엔드포인트(/ai/stream/ws)로 전송한다.

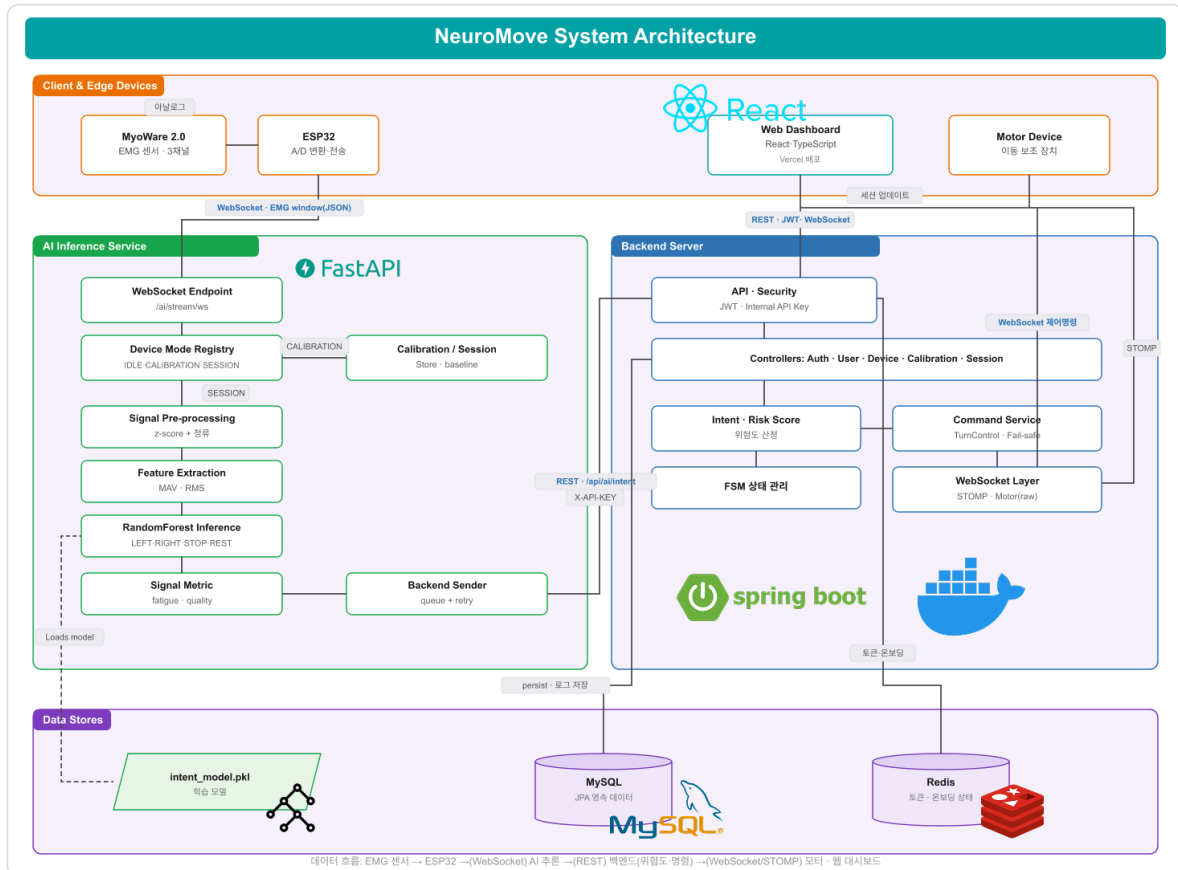
AI 서버는 deviceId를 기준으로 디바이스의 현재 모드(IDLE / CALIBRATION / SESSION)를 판별하여 수신 데이터를 라우팅한다. 주행 세션의 경우 최근 window들을 누적하여 일정 개수(5개)마다 신호 전처리(DC offset 제거 또는 calibration baseline 기반 z-score, 정규), 특징 추출(채널별 MAV·RMS), RandomForest 추론을 수행하고, 동시에 fatigueScore와 signalQuality 지표를 계산한다.

추론 결과는 intent, confidence, fatigueScore, signalQuality를 포함하는 형태로 백엔드 서버의 REST API(POST /api/ai/intent)에 전달된다. 이 전송은 stream 수신 경로를 막지 않도록 백그라운드 큐와 재시도 로직을 통해 비동기로 처리된다.

백엔드 서버는 AI 서버로부터 전달된 intent, confidence, fatigueScore, signalQuality를 기반으로 위험도(Risk Score)를 계산하고, timestamp, 신뢰도, 신호 품질 등을 함께 검증하여 fail-safe 정책을 적용한다. 이후 FSM 상태와 speedLevel을 결정한 뒤 최종 제어 명령(REST, LEFT, RIGHT, STOP, EMERGENCY_STOP, BLOCKED)을 생성한다.

BLOCKED를 제외한 명령은 WebSocket을 통해 모터 디바이스로 전송되어 실제 구동 동작으로 이어지며, 동시에 STOMP 기반 WebSocket을 통해 프론트엔드 대시보드로 AI 추론 결과, 위험도, FSM 상태, 모터 상태가 실시간 전달되어 사용자에게 시각화된다.

6.4. Architecture Diagram



NeuroMove는 임베디드 계층, AI 서버 계층, 백엔드 계층, 프론트엔드 계층으로 구성된 다계층(multi-tier) 아키텍처를 따른다.

- 임베디드 계층: MyoWare 2.0 EMG 센서와 ESP32로 구성되며, 아날로그 신호를 디지털로 변환하여 WebSocket으로 스트리밍한다.
- AI 서버 계층: Python·FastAPI·Uvicorn 기반으로 신호 처리와 의도 추론을 담당한다. scikit-learn RandomForest 모델(intent_model.pkl)을 사용하며, NumPy·Pandas로 신호를 처리한다. Random Forest는 비교적 적은 EMG 데이터에서도 안정적인 분류 성능을 보이고 추론 속도가 빠르기 때문에, 실시간 이동 제어 시스템의 AI 모델로 채택하였다.
- 백엔드 계층: Java 17·Spring Boot 3.2.5 기반으로 인증, 디바이스·세션·캘리브레이션 관리, 위험도 산정, 명령 결정, 모터 제어, 실시간 통지를 담당한다. 데이터 저장에는 MySQL, 온보딩·토큰 등 휘발성 상태에는 Redis를 사용한다.
- 프론트엔드 계층: React·TypeScript·Vite·shadcn/ui 기반의 웹 대시보드로, 실시간 상태 수신은 STOMP 기반 WebSocket으로 처리하고, 사용자 인증·세션 관리·로그 조회 등 비실시간 요청은 REST API(JWT 인증)로 백엔드와 통신하며 Vercel에 배포된다. 백엔드 서버는 AWS EC2 환경에서 Docker로 구동된다.

AI 서버와 백엔드 서버 간 내부 통신은 X-API-KEY(내부 API 키)로 인증되며, 사용자-백엔드 간 통신은 JWT 토큰으로 보호된다.

7. Implementation

7.1. Design Methods

본 프로젝트는 실시간 생체신호 처리와 이동 의도 분석을 위해 다음과 같은 개발 환경을 사용한다.

- Hardware: MyoWare 2.0 EMG Sensor, ESP32
- Embedded: Arduino IDE, C++
- AI Server: Python, FastAPI, NumPy, Pandas, Scikit-learn
- Backend: Java, Spring Boot
- Database: MySQL
- Communication: WebSocket, REST API
- Version Control: Git, GitHub
- Development Tool: Visual Studio Code, IntelliJ IDEA
- Deployment: Docker

AI 모델 학습에는 Random Forest 기반 머신러닝 모델을 사용하며, MAV(Mean Absolute Value)와 RMS(Root Mean Square)를 특징값으로 활용한다.

7.2. Code explanation

본 시스템은 AI 서버, 백엔드 서버, 프론트엔드의 세 저장소로 구성된다. 각 저장소의 핵심 모듈과 구현 내용을 설명한다.

7.2.1 AI 서버 (NeuroMove_AI)

AI 서버는 Python·FastAPI 기반으로 EMG 신호 처리와 사용자 의도 추론을 담당하며, 기능별로 routers, services, schemas, storage, models, training 모듈로 구성된다.

- **main.py:** FastAPI 애플리케이션 진입점으로 **session**, **calibration**, **stream** 라우터를 등록한다. **lifespan**을 통해 백그라운드 **sweeper**를 실행하여, EMG 스트림이 30초 이상 끊긴 디바이스의 점유(lock)를 주기적으로 강제 해제함으로써 lock 누수를 방지한다.
- **routers/stream_router.py:** ESP32와 연결되는 단일 WebSocket 채널(/ai/stream/ws)을 제공한다. ESP32는 캘리브레이션·세션 구분 없이 EMG window만 전송하고, 서버가 **deviceld** 기준으로 모드를 판별하여 처리한다.
- **storage/device_mode_registry.py:** 디바이스별 현재 모드(IDLE, CALIBRATION, SESSION)와 활성 ID를 관리하며, 들어온 데이터를 적절한 서비스로 라우팅하는 기준이 된다.
- **services/signal_processing_service.py:** 원시 신호에서 DC offset을 제거하거나, 캘리브레이션 **baseline**이 있을 경우 채널별 평균·표준편차 기준 **z-score** 정규화를 수행하고, 정류(절댓값)와 선택적 정규화를 적용한다. 3채널(방향용 ch0·ch1, STOP 전용 ch2)을 지원한다. DC offset 제거, Min-Max 정규화, **z-score** 등 여러 전처리 방식을 비교한 결과 REST baseline 기반 **z-score** + 정류 방식이 가장 높은 정확도를 보여 최종 채택하였다.

- `services/feature_service.py`: 전처리된 채널 신호에서 MAV(평균 절댓값)와 RMS(제곱평균제곱근)를 계산하여 `[ch0_mav, ch0_rms, ch1_mav, ch1_rms, ch2_mav, ch2_rms]` 형태의 6차원 특징 벡터를 생성한다.
 - `services/inference_service.py`: 학습된 `RandomForest` 모델(`models/intent_model.pkl`)로 의도를 분류한다. 모델 추론에 앞서 활성화도(activation) 임계값 미만이면 REST로 처리하고, 턱에 부착된 STOP 전용 채널(ch2)이 임계값 이상이면 다른 명령보다 우선하여 STOP으로 판정하는 규칙 기반 안전 로직을 적용한다.
 - `services/signal_metric_service.py`: 신호 품질(signalQuality)과 근육 피로도(fatigueScore) 지표를 계산하여 백엔드의 위험도 산정에 활용되도록 한다.
 - `services/stream_service.py`: 세션 window를 누적하여 일정 개수(5개)마다 추론을 수행하고, 결과를 백그라운드 큐에 적재한다. 별도 워커 스레드가 큐를 소비하여 백엔드로 전송하며, 최대 3회 재시도하여 stream 수신 경로가 막히지 않도록 한다.
 - `training/train_intent_model.py`: REST 구간 baseline 기반 z-score 전처리 후 MAV·RMS 특징을 추출하여 `RandomForestClassifier(n_estimators=100, class_weight=balanced)`를 학습하고 `intent_model.pkl`로 저장한다.
 - AI 모델 학습을 위해 Decision Tree, Support Vector Machine(SVM), Random Forest 등의 머신러닝 기법을 비교하였다. Decision Tree는 모델 구조가 단순하고 해석이 쉽다는 장점이 있으나, 학습 데이터에 과도하게 적합되는 과적합(Overfitting) 문제가 발생하기 쉽다. 또한 EMG 신호와 같이 사용자별 편차가 크고 노이즈가 포함된 데이터에서는 분류 성능이 불안정할 수 있다. SVM은 비교적 적은 데이터에서도 우수한 분류 성능을 보이는 대표적인 분류 알고리즘이다. 그러나 다중 클래스 분류를 위해 추가적인 설정이 필요하며, 하이퍼파라미터(C, Gamma 등)에 따라 성능 변화가 크다. 또한 새로운 데이터가 입력될 때의 추론 과정이 상대적으로 복잡하여 실시간 처리 환경에서는 부담이 될 수 있다.
- 딥러닝 기반 모델(CNN, LSTM)의 경우 EMG 신호의 복잡한 패턴을 학습할 수 있다는 장점이 있지만, 충분한 성능을 확보하기 위해서는 대규모 학습 데이터와 높은 연산 자원이 필요하다. 본 프로젝트는 제한된 수의 사용자 데이터를 활용하며 실시간 의도 분류를 수행해야 하므로 딥러닝 모델을 적용하기에는 데이터 규모와 시스템 자원 측면에서 비효율적이라고 판단하였다. 반면 Random Forest는 여러 개의 Decision Tree를 앙상블 방식으로 결합하여 과적합 문제를 완화할 수 있으며, 노이즈가 포함된 데이터에서도 안정적인 분류 성능을 제공한다. 또한 MAV, RMS와 같은 저차원 특징 벡터만으로도 높은 분류 정확도를 얻을 수 있고, 학습 및 추론 속도가 빠르다는 장점이 있다. 본 프로젝트에서는 3채널 EMG 신호에서 추출한 MAV와 RMS 특징을 입력으로 사용하였으며, 실제 실험 과정에서 Decision Tree와 SVM 대비 보다 안정적인 분류 결과를 확인할 수 있었다. 또한 실시간 의도 분류와 경량화된 시스템 구성이 중요하다는 점을 고려하여 최종적으로 RandomForestClassifier를 채택하였다.

7.2.2 백엔드 서버 (NeuroMove_backend)

백엔드 서버는 Java 17·Spring Boot 3.2.5 기반으로, 도메인 중심(domain) 구조로 auth, device, calibration, session, intent, command, fsm, user, websocket 패키지를 구성한다. JPA(MySQL), Spring Security(JWT), Redis, WebSocket, Swagger를 사용한다.

- domain/auth: JWT 기반 인증과 회원가입·로그인·토큰 갱신·온보딩을 처리하며, 온보딩 임시 상태는 Redis에 저장한다.
- domain/intent: AI 서버의 추론 결과를 수신하는 핵심 도메인이다. IntentService는 위험도(Risk Score)를 $0.4 \times \text{피로도} + 0.4 \times (1 - \text{신호품질}) + 0.2 \times \text{주행지속비율}$ 로 계산하고, speedLevel(0~5)을 산정하며 최종 명령을 결정한다.
- 페일세이프(fail-safe) 정책: 명령이 3초 이상 지연(stale)되면 연속 3회 누적 시 자동 STOP, 신호 품질이 0.2 미만인 이상 패턴이 연속 3회 발생하면 EMERGENCY_STOP, confidence가 0.5 미만이면 BLOCKED, 위험도가 0.7 이상이면 명령을 BLOCKED로 차단하고 FSM 상태를 EMERGENCY_STOP으로 전환하여 안전을 최우선으로 보장한다.
- domain/command: TurnControlManager가 회전 제어를 담당한다. LEFT/RIGHT가 연속 2회 감지되면 회전을 확정하여 즉시 전송하고, 약 500ms 후 자동으로 FORWARD를 전송하여 과도한 회전을 방지한다. STOP·EMERGENCY_STOP은 즉시 반영된다.
- domain/fsm: 주행 상태 기계(DRIVING, FATIGUE_COMPENSATING, EMERGENCY_STOP 등)를 관리하여 위험도에 따른 상태 전이를 기록한다.
- domain/calibration, domain/session, domain/device: 사용자별 캘리브레이션 프로파일, 주행 세션, EMG·모터 디바이스 정보를 관리하며, AI 서버와는 별도의 AiCalibrationClient·AiSessionClient를 통해 연동한다.
- domain/websocket: 모터 디바이스와의 raw WebSocket 통신(MotorWebSocketHandler) 및 프론트엔드와의 STOMP 기반 실시간 통지(SessionWebSocketService)를 처리한다. AI 서버 내부 호출은 InternalApiKeyFilter(X-API-KEY)로 인증한다.

명령	발생 주체	저장 위치	설명
REST / LEFT / RIGHT / STOP	AI 추론 결과	DB + 모터 전송	정상 판단 시 그대로 전달
EMERGENCY_STOP	백엔드 생성	DB + 모터 전송	이상 패턴 3회 연속 시 발동
BLOCKED	백엔드 생성	DB에만 기록	confidence 낮음, stale, 위험도 0.7 이상 시 모터 전송 차단
FORWARD	백엔드 생성	모터 전송만	LEFT/RIGHT 확정 후 500ms 뒤 자동 직진 복귀
FINISH	백엔드 생성	모터 전송만	세션 종료 시 모터 정지 신호

7.2.3 프론트엔드 (NeuroMove_frontend)

프론트엔드는 **React·TypeScript·Vite** 기반의 단일 페이지 애플리케이션(SPA)으로, 고성능의 사용자 인터페이스와 실시간 데이터 시각화를 제공한다. **shadcn/ui**와 **Tailwind CSS**를 통해 직관적인 UX를 구축하고 **Vercel**을 통해 지속적 배포(CD)를 수행한다.

- 컴포넌트 중심 UI/UX (pages): 로그인(Login), 회원가입(Signup), 디바이스 등록(SignupDevices), 메인 대시보드(Main), 로그 목록·상세(Logs, LogDetail), 마이페이지(MyPage), 그리고 단계별 캘리브레이션(Start, Step2~Step5, Result) 화면으로 구성된다.
- 통신 및 상태 관리 (lib/api.ts): TanStack Query 기반의 상태 관리를 적용했으며, 공통 API 요청 함수를 설계하여 통신 로직을 모듈화했다. 매 요청마다 JWT 토큰을 자동으로 주입하고 백엔드의 규격화된 응답 (success/code/message/data)에 대한 에러 핸들링을 통합적으로 수행한다.
- 실시간 스트리밍 처리 (lib/websocket.ts): 외부 라이브러리에 의존하지 않고, STOMP-over-WebSocket 클라이언트를 직접 구현하여 기술적 경량화를 달성했다. 백엔드의 /ws 엔드포인트와 상시 연결을 유지하며, /topic 구독을 통해 intent, riskScore, command, speedLevel의 세션 데이터를 실시간으로 수신한다.
- 데이터 시각화 (components/Waveform.tsx): 실시간 신호 및 상태를 시각화하는 컴포넌트이며, AppSidebar와 PageTransition을 적용하여 일관된 레이아웃 구조와 매끄러운 화면 전환 경험을 제공한다.

8. Testing

Test Case Name	SIGNAL_COLLECTION
Test Case ID	NEUROMOVE_TEST_1
Test Case Description	EMG 센서가 사용자의 근육 신호를 정상적으로 수집하고 AI 서버까지 전달할 수 있는지 확인한다.
Test Steps	MyoWare EMG 센서를 사용자의 근육에 부착한다. ESP32를 통해 EMG 신호를 수집한다. 수집된 신호를 WebSocket을 이용하여 AI 서버로 전송한다. AI 서버에서 수신된 데이터의 채널 수, 데이터 길이 및 순서(sequence number)를 확인한다. 수신 데이터와 송신 데이터의 일치 여부를 검증한다.
Expected Result	EMG 신호가 정상적으로 수집된다. 데이터 손실 없이 AI 서버에 전달된다. 모든 채널 데이터가 정상적으로 저장된다. 실시간 신호 수집 환경에서 안정적으로 동작한다.
Test Case Name	CALIBRATION

Test Case ID	NEUROMOVE_TEST_2
Test Case Description	사용자별 캘리브레이션이 정상적으로 수행되어 baseline 과 임계값이 생성되는지 확인한다.
Test Steps	캘리브레이션을 시작하여 디바이스 모드를 CALIBRATION 으로 전환한다. REST, LEFT, RIGHT, STOP 단계별로 EMG 신호를 수집한다. 각 단계 종료 후 채널별 평균·표준편차 및 임계값을 계산한다. 캘리브레이션 종료(finish) 시 프로파일이 저장되는지 확인한다.
Expected Result	단계별 신호가 정상적으로 누적된다. 채널별 baseline(mean/std) 과 임계값이 생성된다. 생성된 프로파일이 세션 추론에 적용된다.

Test Case Name	INTENT_INFERENCE
Test Case ID	NEUROMOVE_TEST_3
Test Case Description	AI 서버가 EMG 특징 벡터로부터 의도를 정확하게 분류하는지 확인한다.
Test Steps	주행 세션을 시작하여 디바이스 모드를 SESSION 으로 전환한다. LEFT, RIGHT, STOP, REST 동작에 해당하는 신호를 입력한다. 전처리·특징 추출(MAV, RMS) 후 RandomForest 추론을 수행한다. 예측된 intent 와 confidence 가 기대값과 일치하는지 검증한다.
Expected Result	각 동작에 대해 올바른 intent 가 분류된다. STOP 전용 채널 활성화 시 STOP 이 우선 적용된다. 신호가 약한 경우 REST 로 처리된다.

Test Case Name	FAIL_SAFE
Test Case ID	NEUROMOVE_TEST_4
Test Case Description	비정상 상황에서 페일세이프 정책이 안전 명령을 우선 적용하는지 확인한다.
Test Steps	confidence 가 0.5 미만인 추론 결과를 전송한다. signalQuality 가 0.2 미만인 이상 패턴을 연속 3회 전송한다. 명령 타임스탬프가 3초 이상 지연된 stale 상황을 연속 3회 발생시킨다. 각 상황에서 백엔드가 결정한 최종 명령을 확인한다.

Expected Result	낮은 confidence는 BLOCKED로 처리된다. 이상 패턴 3회 연속 시 EMERGENCY_STOP으로 전환된다. stale 3회 누적 시 자동 STOP이 적용된다.
------------------------	---

Test Case Name	TURN_CONTROL
Test Case ID	NEUROMOVE_TEST_5
Test Case Description	회전 제어 로직이 연속 감지 및 자동 직진 복귀를 올바르게 수행하는지 확인한다.
Test Steps	LEFT 의도를 연속으로 전송한다. 연속 2회 감지 시 회전 명령이 모터로 전송되는지 확인한다. 약 500ms 후 자동 FORWARD가 전송되는지 확인한다. 동일 방향이 반복 전송될 때 중복 실행이 억제되는지 확인한다.
Expected Result	연속 2회 감지 시 회전이 1회 확정 전송된다. 회전 후 자동으로 직진(FORWARD)이 전송된다. FORWARD 이전까지 동일 방향 재실행이 차단된다.

Test Case Name	END_TO_END
Test Case ID	NEUROMOVE_TEST_6
Test Case Description	EMG 수집부터 모터 제어 및 화면 표시까지 전체 파이프라인이 동작하는지 확인한다.
Test Steps	ESP32에서 EMG 신호를 WebSocket으로 전송한다. AI 서버가 추론 결과를 백엔드로 전달한다. 백엔드가 위험도·명령을 결정하여 모터와 프론트엔드로 전송한다. 웹 대시보드에서 intent, riskScore, command가 실시간 갱신되는지 확인한다.
Expected Result	전체 경로에서 데이터 손실 없이 명령이 생성된다. 모터 디바이스에 제어 명령이 전달된다. 프론트엔드에 실시간 상태가 표시된다.

9. Results

9.1. Results

구현된 시스템에 대해 AI 모델 성능 평가와 통합 동작 검증을 수행하였다. 학습 데이터는 3채널 EMG window 기준 총 1,920건(LEFT, RIGHT, REST/NEUTRAL, STOP/CLENCH 약 480건씩)으로 구성되며, 70%를 학습, 30%를 테스트에 사용하였다.

RandomForest 분류 모델의 테스트 정확도는 약 96.7%로 측정되었으며, 클래스별 정밀도(Precision)·재현율(Recall)·F1-score는 다음과 같다.

Class	Precision	Recall	F1-score	Support
LEFT	0.95	0.97	0.96	144
REST	0.95	0.94	0.95	144
RIGHT	0.97	0.98	0.98	145
STOP	0.99	0.97	0.98	143
Accuracy	-	-	0.97	576

또한 통합 검증에서는 ESP32에서 전송된 EMG 스트림이 AI 서버의 추론을 거쳐 백엔드의 위험도 산정 및 명령 결정으로 이어지고, 모터 디바이스와 웹 대시보드까지 실시간으로 반영되는 end-to-end 동작을 확인하였다. AI 서버는 약 5개 window마다(설계상 약 0.5초 주기) 추론을 수행하며, 백엔드의 파일세이프 정책(BLOCKED, STOP, EMERGENCY_STOP)이 비정상 입력에 대해 정상적으로 발동함을 확인하였다.

9.2. Result Analysis

전처리 방법 비교 및 선택 근거

신호 전처리 방식이 분류 정확도에 미치는 영향을 확인하기 위해, 동일한 학습 데이터(3채널 EMG window 1,920건)와 동일한 분류기(RandomForest, n_estimators=100), 동일한 학습·테스트 분할(7:3, random_state=42) 조건에서 대표적인 전처리 방법을 비교 실험하였다. 결과는 다음과 같다.

전처리 방법	Accuracy	Macro F1-score
DC offset 제거 + 정류	43.58%	0.435
정규화(Min-Max) + 정류	30.90%	0.310
REST baseline z-score + 정류 (채택)	96.70%	0.967

단순 DC offset 제거나 Min-Max 정규화는 window 단위로 신호를 처리하는 과정에서 EMG 신호의 절대 크기 정보를 상당 부분 제거하게 된다. EMG 기반 의도 분류에서는 각 채널의 활성화 차이가 중요한 특징으로 작용하는데, 이러한 전처리 방식은 채널 간 상대적인 활성화 차이를 감소시켜 방향 의도를 구분하는 데 필요한 정보를 손실시키는 문제가 발생하였다. 그 결과 정확도가 약 30~44% 수준으로 크게 감소하였으며, 특히 LEFT와 RIGHT 클래스 간 오분류가 빈번하게 발생하였다.

반면 사용자의 **REST**(휴지) 구간에서 측정한 채널별 평균과 표준편차를 기준으로 정규화하는 **z-score + 정류** 방식은 사용자의 개인별 신호 특성을 반영하면서도 채널 간 활성화도 차이를 유지할 수 있었다. 또한 사용자의 기준 상태를 중심으로 신호를 해석하기 때문에 센서 부착 위치의 차이, 피부 상태, 근육 크기 등의 개인차에 의한 영향을 줄일 수 있었다. 실험 결과 해당 방식은 **Accuracy 96.70%, Macro F1-score 0.967**로 가장 우수한 성능을 보였으며, 캘리브레이션 과정에서 생성된 **baseline** 정보를 추론 단계에서도 그대로 활용할 수 있다는 점에서 실용성 또한 높다고 판단하였다. 따라서 본 프로젝트에서는 **z-score + 정류** 방식을 최종 전처리 기법으로 채택하였다.

실험 결과 **Random Forest** 모델은 **LEFT, RIGHT, STOP, REST** 네 가지 의도를 약 **96.7%**의 높은 정확도로 분류하였다. 특히 **STOP** 클래스는 전용 채널(**ch2**)을 별도로 구성하고 규칙 기반 우선 처리 로직을 함께 적용함으로써 **0.98** 수준의 높은 **F1-score**를 달성하였다. 이는 사용자가 긴급하게 이동을 중단해야 하는 상황에서도 시스템이 해당 의도를 안정적으로 인식할 수 있음을 의미한다. 이동 보조 시스템에서 **STOP** 명령은 안전과 직결되는 기능이므로, 높은 **STOP** 분류 성능은 본 시스템의 중요한 성과 중 하나라고 할 수 있다.

LEFT와 **RIGHT** 클래스 역시 높은 정밀도와 재현율을 보였으며, 이는 방향 제어를 위해 각각 독립적인 근육 사용 패턴을 설계하고 채널별 특징을 효과적으로 추출한 결과로 판단된다. 특히 **MAV**와 **RMS** 특징은 계산량이 적으면서도 근육 활성화도의 크기를 잘 반영하여 실시간 시스템에 적합한 특징값임을 확인할 수 있었다.

REST 클래스의 재현율이 상대적으로 낮게 나타난 것은 사용자가 의도적으로 움직이지 않는 상태에서도 미세한 근육 수축이나 센서 노이즈가 발생할 수 있기 때문이다. 이러한 신호는 방향 의도를 나타내는 약한 활성 신호와 유사한 형태를 가질 수 있어 일부 오분류가 발생하였다. 이를 해결하기 위해 활성화도 임계값(**activation threshold**)을 적용하여 일정 수준 이하의 신호는 **REST** 상태로 처리하도록 하였으며, 사용자별 캘리브레이션 정보를 활용하여 개인차에 따른 오분류 가능성을 최소화하였다.

또한 본 프로젝트는 단순히 의도를 분류하는 데 그치지 않고, 위험도(**Risk Score**)를 기반으로 최종 제어 명령을 결정하는 다단계 안전 구조를 적용하였다. 신호 품질(**signal quality**), 근육 피로도(**fatigue score**), 주행 지속시간 등의 요소를 종합하여 위험도를 계산하고, 위험 수준에 따라 속도 단계를 조정하거나 **EMERGENCY_STOP**을 수행하도록 설계하였다. 이를 통해 단일 추론 오류가 곧바로 위험한 이동 명령으로 이어지는 상황을 방지할 수 있었다.

특히 **confidence** 값이 낮은 경우 **BLOCKED** 상태로 전환하거나, 신호 품질이 지속적으로 저하되는 경우 자동 정지를 수행하는 다중 페일세이프(**Fail-Safe**) 정책을 적용하였다. 이러한 구조는 실제 사용 환경에서 발생할 수 있는 센서 오류, 통신 지연, 사용자의 피로 누적 등의 다양한 상황에 대응하기 위한 안전 장치로서 효과적으로 동작하였다.

종합적으로 본 실험 결과는 **EMG** 신호 기반 의도 분류와 위험도 기반 제어 구조가 실제 이동 보조 시스템에 적용 가능한 수준의 성능과 안정성을 가질 수 있음을 보여준다. 또한 사용자별 캘리브레이션과 다중 안전 계층을 결합함으로써 단순 분류 시스템을 넘어 실시간 이동 제어 시스템으로 확장할 수 있는 가능성을 확인하였다.

한계 및 향후 개선 방향

- 현재 시스템은 **EMG** 신호 기반 의도 분류와 실시간 제어 파이프라인의 구현 및 검증에 중점을 두고 개발되었다. 향후에는 다양한 사용자로부터 수집한 실제 **EMG** 데이터를 활용하여 모델을 추가 학습하고 성능을 더욱 향상시킬 수 있을 것으로 기대된다.
- 사용자마다 근육 특성과 신호 패턴에 차이가 존재하므로, 보다 정밀한 개인 맞춤형 제어를 위해 캘리브레이션 절차를 개선하고 사용자 특성에 적응하는 방식의 제어 기법을 적용할 수 있을 것으로 판단된다.
- 현재 구현된 시스템은 실시간 의도 추론과 제어 명령 생성이 가능함을 확인하였으며, 향후 실제 이동 보조 장치와의 연동 실험을 통해 응답 속도와 안정성을 추가적으로 검증할 계획이다.
- 또한 다양한 연령대와 사용자 환경을 대상으로 사용성 평가를 수행하여 시스템의 편의성과 신뢰성을 높이고, 실제 활용 환경에 적합한 방향으로 발전시킬 예정이다.

10. Team and Process

10.1. Design Schedule

프로젝트는 2026년 1학기(3월~6월) 동안 진행되었으며, 단계별 주요 일정은 다음과 같다.

단계	기간	주요 활동	산출물
요구사항 분석	3월	문제 정의, 선행 연구 조사, 요구사항 정의, 기술 스택 선정	기획서, 요구사항 명세
시스템 설계	3월~4월	아키텍처 설계, 데이터 포맷·API 설계, 디바이스 구성 설계	설계 문서, API 명세
하드웨어·신호 처리	4월	EMG 센서·ESP32 연동, 신호 수집·전처리·특징 추출 구현	신호 처리 모듈
AI 모델 개발	4월~5월	데이터 수집, 특징 설계, RandomForest 학습·평가	학습 모델, 추론 서버
백엔드·프론트 개발	4월~5월	인증·세션·캘리브레이션·명령 도메인, 웹 대시보드 구현	백엔드·프론트엔드
통합 및 테스트	5월~6월	end-to-end 통합, 페일세이프·회전 제어 검증, 버그 수정	테스트 결과
최종 발표·문서화	6월	결과 분석, 보고서 작성, 발표 준비	최종 보고서, 발표 자료

단계	3월				4월				5월				6월		
	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15
요구사항 분석															
시스템 설계															
하드웨어·신호 처리															
AI 모델 개발															
백엔드·프론트 개발															
통합 및 테스트															
최종 발표·문서화															

10.2. Team Member Task Assignment

본 프로젝트는 4인(이연우, 박수연, 박수빈, 박원)이 AI 서버, 백엔드, 프론트엔드, 임베디드·하드웨어 영역을 분담하여 진행하였다.

팀원	담당 영역	주요 역할
이연우	AI 서버 / 시스템 통합	[신호 처리·특징 추출·추론] 파이프라인 구현, 백엔드-AI 서버 연동, 도메인 설계 및 연동 구현
박수연	AI 서버 / 하드웨어	EMG 센서 데이터 수집 및 실시간 전송 시스템 구현, 신호 전처리·특징 추출(MAV, RMS) 설계, AI 기반 의도 분류 모델 설계 및 학습
박수빈	백엔드 / 하드웨어	Refresh Token 인증, 온보딩 연동, Redis 기반 디바이스 관리, 사용자 상태·운행 로그 관리, ESP32 펌웨어 및 모터 제어 WebSocket 연동, 타임아웃 STOP·이상 패턴 EMERGENCY_STOP 처리
박원	백엔드 / 프론트엔드	웹 대시보드, 캘리브레이션·세션 화면 개발, 위험도·페일세이프 로직 구현, 프론트엔드 전체 API/WebSocket(STOMP) 연동, 백엔드 Calibration·세션·AI 추론 도메인 API 개발, AWS 백엔드 배포 및 Vercel 프론트 배포, 통합 E2E 테스트 및 QA

10.3. BudgetsZ

주요 비용은 **EMG** 측정용 하드웨어 구매 비용으로, 개발 단계에서는 로컬·무료 티어 환경을 활용하여 서버 운영 비용을 최소화하였다. 아래 금액은 예상치이며 구매처와 수량에 따라 달라질 수 있다.

항목	수량	예상 비용(KRW)
MyoWare 2.0 EMG 센서	3	약 150,000원
EMG 전극 패드(소모품)	다수	1회 측정 시 약 30,000원
ESP32 개발 보드	1~2	약 20,000원
모터 / 구동부·연결 부품	1식	약 50,000원
기타(케이블, 브레드보드 등)	1식	약 20,000원
서버 운영(개발 단계 로컬·무료 티어)	2	약 40,000원
합계(예상)	-	약 310,000원

향후 실제 스마트 휠체어 또는 이동 보조 장치와 연동하여 상용화를 고려할 경우, 모터 제어 장치, 배터리 시스템, 추가 센서 및 클라우드 서버 비용이 추가로 발생할 수 있다.

11. Conclusion

본 프로젝트는 저비용 **EMG** 센서와 **ESP32**를 기반으로 사용자의 근전도 신호로부터 이동 의도를 실시간으로 추론하고, 이를 이동 보조 장치 제어로 연결하는 스마트 이동 제어 시스템(**NeuroMove**)을 설계하고 구현하였다. 시스템은 **AI** 서버(**FastAPI**), 백엔드 서버(**Spring Boot**), 프론트엔드(**React**)의 세 계층으로 구성되며, 신호 수집부터 의도 추론, 위험도 기반 명령 결정, 모터 제어 및 실시간 시각화까지 이어지는 **end-to-end** 파이프라인을 구현하였다.

AI 모델은 **MAV·RMS** 특징과 **Random Forest** 분류기를 활용하여 **LEFT, RIGHT, STOP, REST** 네 가지 의도를 약 **96.7%**의 정확도로 분류하였으며, 백엔드 서버에서는 추론 결과를 바탕으로 위험도 산정, **FSM** 상태 제어 및 다중 페일세이프(**Fail-Safe**) 정책을 적용하여 최종 제어 명령의 안정성을 높였다. 또한 **ESP32**와 **AI** 서버 간 실시간 **WebSocket** 통신을 통해 **EMG** 데이터를 지연 없이 처리할 수 있도록 구현하였으며, 사용자별 캘리브레이션 기능을 제공하여 개인의 근육 특성 차이를 반영할 수 있도록 설계하였다.

특히 **STOP** 전용 채널의 규칙 기반 우선 처리, 활성화 임계값 기반 **REST** 처리, 위험도 산정과 다중 페일세이프 정책을 결합하여 안전성을 다층적으로 확보하였다. 또한 회전 제어 로직과 디바이스 연결 상태 관리, 세션 관리 기능 등을 구현하여 실제 사용 환경에서도 안정적으로 동작할 수 있는 시스템 구조를 구축하였다. 이를 통해 단순한 의도 분류 모델 개발을 넘어, 실시간 이동 제어 시스템으로서의 가능성을 확인할 수 있었다.

프로젝트 수행 과정에서 **EMG** 신호의 개인차와 노이즈 문제, 제한된 학습 데이터로 인한 일반화 성능 확보의 어려움 등 여러 한계점도 확인할 수 있었다. 특히 센서 부착 위치나 사용자 상태에 따라 신호 특성이 달라질 수 있기 때문에 다양한 사용자를 대상으로 한 추가 데이터 수집과 모델 검증이 필요함을 확인하였다. 또한 현재는 시뮬레이션 및 프로토타입 환경에서 검증을 수행하였으므로 실제 이동 보조 장치와 연동한 장기적인 실험이 추가적으로 요구된다. 더불어 **EMG** 신호와 제어 명령은 정상적으로 수신되었으나, 실험에 사용한 저가형 모터 키트의 조립 정밀도 한계로 인해 한쪽 바퀴에 유격이 발생하였고, 이로 인해 실제 주행 과정에서 직진 및 방향 전환의 제어 정확도가 저하되는 문제가 확인되었다.

향후에는 실제 사용자 **EMG** 데이터를 활용한 모델 재학습과 강건성 향상, 캘리브레이션 절차 고도화, 실제 모터 구동 환경에서의 지연 시간 및 안전성 검증, 그리고 다양한 사용자 대상 사용성 평가를 통해 시스템을 발전시킬 계획이다. 또한 **Random Forest** 기반 모델뿐만 아니라 **CNN**, **LSTM** 등 딥러닝 기반 모델과의 성능 비교를 수행하여 보다 정교한 의도 인식 기술을 연구할 예정이다.

본 시스템은 상지 사용이 제한된 사용자의 이동권 향상과 자율성 증대에 기여할 수 있으며, 향후 스마트 휠체어, 재활 로봇, 의수·의족 제어, 스마트 헬스케어 등 다양한 분야로 확장 가능한 기반 기술로서 의의를 가진다. 나아가 생체신호 기반 인간-기계 인터페이스 기술의 실용화를 위한 하나의 사례를 제시하였으며, 사용자 중심의 지능형 이동 보조 시스템 개발에 활용될 수 있을 것으로 기대된다.

Reference

- MyoWare 2.0 Muscle Sensor, SparkFun Electronics. <https://www.sparkfun.com/>
- Espressif Systems, ESP32 Technical Reference. <https://www.espressif.com/>
- FastAPI Documentation. <https://fastapi.tiangolo.com/>
- Spring Boot Reference Documentation. <https://spring.io/projects/spring-boot>
- scikit-learn: RandomForestClassifier. <https://scikit-learn.org/>
- React Documentation. <https://react.dev/>
- M. B. I. Reaz, M. S. Hussain, F. Mohd-Yasin, Techniques of EMG signal analysis: detection, processing, classification and applications, Biological Procedures Online, 2006.